



UNIVERSIDAD NACIONAL DEL SUR

PRINCIPIOS DE COMPUTADORAS II

Proyecto N°1

Space Invaders - Diseño Orientado a Objetos-

Integrante:

Antonella Malaspina

Destinatarios:

Profesor: Dr. Juan Miguel Bajo

Asistente: Ing. Alejandro Zambrano

Ayudante: Ing. Jorge Iván Lopez Perez

Bahía Blanca, Buenos Aires

Fecha de entrega:

11 de noviembre de 2025

Índice

1. Introducción al Diseño del Sistema	2
2. Descripción General del Sistema	2
3. Diagrama UML de Clases	3
3.1. Relaciones Principales	5
3.2. Descripción de las Clases	5
3.2.1. Clases Base Abstractas	5
3.2.2. Clases de Control del Sistema	7
3.2.3. Clases de Entidades del Juego	11
3.2.4. Nave_enemiga	15
4. Interacciones entre Clases	16
5. Justificación del Diseño	16
6. Conclusión	17

1. Introducción al Diseño del Sistema

El presente informe describe el diseño orientado a objetos (POO) y la arquitectura modular implementados para el desarrollo del videojuego Space Invaders en C++ utilizando Qt.

La estructura del sistema se fundamenta en un diagrama UML de clases que modela las entidades, sus comportamientos y las relaciones entre los distintos módulos. Este diagrama no solo proporciona una representación estructurada de cómo se distribuyen las responsabilidades y el flujo de datos, sino que también garantiza la reutilización de código, la claridad y la escalabilidad del proyecto.

El diseño del juego se basa en principios de POO como la herencia, la abstracción y el encapsulamiento. Esto se logra definiendo clases base clave como `Dibujable_Abstracto` (para la representación gráfica) y `Colisionable` (para la gestión de impactos), cuyas funcionalidades se combinan a través de herencia múltiple en entidades esenciales como `Nave`, `Proyectil` y `Alien`.

La reutilización se extiende a los enemigos: las clases `Pulpo`, `Cangrejo` y `Calamar` se especializan a partir de la clase base `Alien`, permitiendo modelar distintos enemigos sin duplicar código.

Finalmente, la lógica central del juego reside en la clase `GameLogic`, que administra objetos, movimiento y colisiones, mientras que `GameWidget` actúa como la capa de interacción con el usuario y gestiona la actualización gráfica a través de Qt. El objetivo es analizar esta arquitectura robusta y modular en detalle.

2. Descripción General del Sistema

El videojuego consiste en una nave controlada por el jugador, capaz de moverse horizontalmente y disparar proyectiles mediante la barra espaciadora. Con la tecla `ESC` el usuario puede pausar la partida.

Los enemigos están organizados en hileras con los siguientes puntajes:

- **Pulpo:** otorga 10 puntos
- **Cangrejo:** otorga 20 puntos
- **Calamar:** otorga 30 puntos
- **Nave enemiga:** otorga 50 puntos (aparece aleatoriamente)

Las barreras otorgan protección parcial al jugador, degradándose a medida que reciben impactos. La lógica central del juego determina los movimientos, colisiones, creación de enemigos y actualización del puntaje.

3. Diagrama UML de Clases

El diseño arquitectónico del sistema se representa mediante el diagrama UML mostrado en la Figura 1. Este diagrama ilustra la jerarquía de clases, las interfaces y las relaciones de dependencia entre los componentes del sistema.

3.1. Relaciones Principales

El diseño se basa en los siguientes tipos de relaciones:

- **Herencia simple:** Pulpo, Cangrejo y Calamar heredan de Alien
- **Herencia múltiple:** Los objetos del juego heredan de Dibujable_Abstracto, Colisionable y Movil según corresponda:
 - Nave: hereda de Dibujable_Abstracto, Colisionable y Movil
 - Alien: hereda de Dibujable_Abstracto, Colisionable y Movil
 - Proyectil: hereda de Dibujable_Abstracto, Colisionable y Movil
 - Barrera: hereda de Dibujable_Abstracto y Colisionable
 - Nave_enemiga: hereda de Dibujable_Abstracto, Colisionable y Movil
- **Composición y Agregación:**
 - GameLogic administra múltiples entidades mediante listas de punteros
 - Enemigos gestiona la creación y administración de aliens
 - GameWidget contiene y coordina GameLogic y Enemigos
- **Dependencia:** GameWidget depende de Tiempo_proceso_Qpainter para la renderización

3.2. Descripción de las Clases

3.2.1. Clases Base Abstractas

Las siguientes clases definen las interfaces fundamentales del sistema:

Dibujable_Abstracto Rol: Es la clase base abstracta que define los atributos geométricos y la interfaz gráfica mínima de cualquier objeto que pueda dibujarse en pantalla.

Responsabilidad: Estandarizar la estructura de los objetos gráficos del juego (posición, tamaño y vida) y garantizar la existencia de un método de dibujo.

Atributos:

- vida: int - cantidad de vidas del objeto
- x, y: float - posición del objeto en el espacio 2D
- ancho, alto: float - dimensiones del sprite

Métodos clave:

- get_x(): float, get_y(): float - obtener posición
- get_ancho(): float, get_alto(): float - obtener dimensiones
- Dibujar(QPainter&): void* - método puro virtual para renderizar el objeto

Relaciones: Es la base de todos los objetos visibles. Implementada por: Alien, Nave, Proyectoil, Barrera, Nave_enemiga.

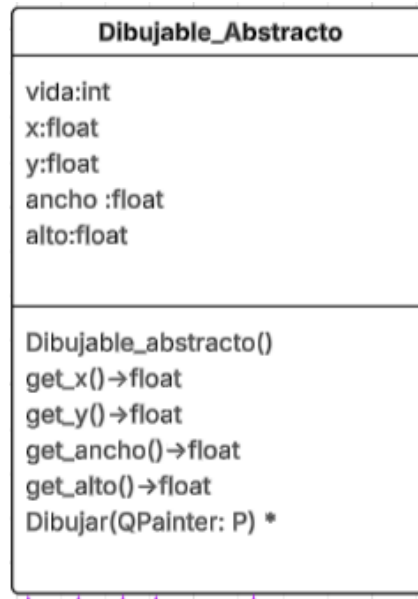


Figura 2: Clase Dibujable_Abtracto

Colisionable Rol: Define la interfaz de detección de colisiones mediante áreas rectangulares.

Responsabilidad: Brindar el mecanismo común para obtener áreas de colisión (QRectF) y evaluar choques entre objetos mediante intersección de rectángulos.

Atributos:

- colisionado: bool - indica si el objeto ya colisionó

Métodos:

- get_area(): QRectF* - método puro virtual que retorna el área de colisión
- CheckCollision(const colisionable* const *bool) - usa intersects() entre rectángulos
- get_colision(): bool - consulta el estado de colisión
- set_colision(bool): void - establece el estado de colisión

Relaciones: Todos los objetos colisionables derivan de esta clase. Implementada por: Alien, Nave, Proyectoil, Nave_enemiga, Barrera.

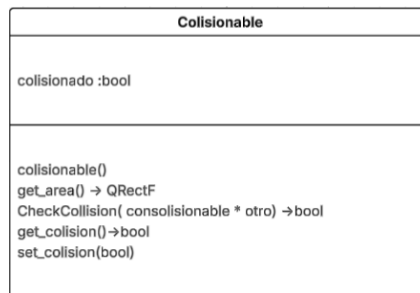


Figura 3: Clase Colisionable

Movil Rol: Interfaz abstracta que define comportamiento de movimiento para objetos dinámicos.

Responsabilidad: Proporcionar una representación independiente para velocidad y movimiento, separada de las responsabilidades de dibujo y colisión.

Atributos:

- `Vx: float` - velocidad horizontal
- `Vy: float` - velocidad vertical

Métodos:

- `movimiento(): void*` - método puro virtual para actualizar la posición
- `set_Vx(float): void` - establece velocidad horizontal
- `set_Vy(float): void` - establece velocidad vertical

Relaciones: La lógica de movimiento se unifica bajo esta clase. Implementada por: `Nave`, `Alien`, `Proyectil`, `Nave_enemiga`.

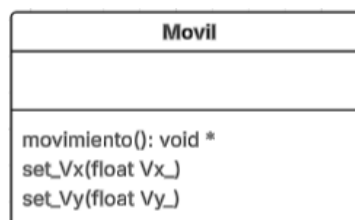


Figura 4: Clase Movil

3.2.2. Clases de Control del Sistema

GameWidget Rol: Clase principal que coordina la interfaz gráfica y el flujo del juego.

Responsabilidades principales:

- Gestionar el ciclo de renderizado mediante `Tiempo_proceso_Qpainter`
- Coordinar la lógica del juego a través de `GameLogic`
- Administrar la creación de enemigos mediante `Enemigos`

-
- Manejar eventos de teclado y mouse del usuario
 - Procesar el loop principal del juego

Atributos principales:

- `timer_principal`: `Qpainter` - temporizador para actualización de pantalla
- Referencias a `GameLogic` y `Enemigos`
- `pausa`: `bool` - estado de pausa del juego
- `juego_iniciado`: `bool` - indica si el juego comenzó

Métodos clave:

- `paintEvent()`: `void` - renderiza la escena
- `keyPressEvent(QKeyEvent*)`: `void` - maneja teclas presionadas
- `keyReleaseEvent(QKeyEvent*)`: `void` - maneja teclas liberadas
- `actualizar_juego()`: `void` - actualiza el estado del juego
- `pausar_bool()`: `bool` - consulta estado de pausa

GameLogic Rol (central del sistema): Es el núcleo del videojuego, que orquesta todos los subsistemas y mantiene el ciclo general del juego.

Responsabilidades principales:

- Inicializar el juego y configurar el estado inicial
- Crear y administrar todos los objetos del juego
- Mover entidades según sus velocidades
- Detectar y resolver colisiones entre objetos
- Generar enemigos y proyectiles dinámicamente
- Controlar el menú principal y el sistema de pausa
- Gestionar puntaje, vidas y condiciones de victoria/derrota
- Administrar la nave bonus y su generación aleatoria

Atributos principales:

- `nave_principal`: `Nave*` - puntero a la nave del jugador
- `lista_barreras`: `Qlists<barrera*>` - lista de barreras activas
- `lista_gameovers`: `Qlists<dibujables*>` - elementos de pantalla de game over
- `lista_proyectil`: `Qlists<proyectil*>` - proyectiles activos en el juego
- `proyectil_nave`: `Qlists<proyectil*>` - proyectiles de la nave del jugador
- `lista_a_dibujar`: `Qlists<Dibujable*>` - todos los objetos a renderizar
- `lista_a_mover`: `Qlists<Movil*>` - todos los objetos móviles

-
- `nave_bonus: Nave_enemiga*` - nave especial que otorga puntos extra
 - `intervalo_disparo_alien: int` - control de frecuencia de disparo enemigo
 - `margen_derecha: int, margen_izquierda: int` - límites de movimiento
 - `contacto_barrera_alien: int` - contador de impactos
 - `cantidad_aliens_caidos: int` - aliens eliminados
 - `direccion_alien_derecha: bool` - dirección actual del grupo de aliens
 - `pausa: bool` - estado de pausa del juego

Métodos clave:

- `inicializar_juego(): void` - configura el estado inicial
- `actualizar(): void` - ciclo principal del juego
- `Dibujar(QPainter&): void` - renderiza todos los objetos
- `Verificar_colisiones(): void` - detecta y procesa todas las colisiones
- `Mover_enemigos(): void` - actualiza posiciones de aliens
- `Disparar_aliens(): void` - genera proyectiles enemigos
- `Generar_nave_bonus(): void` - crea la nave especial aleatoriamente
- `Pressbutton(Qt::key): void` - procesa tecla presionada
- `Releasebutton(Qt::key): void` - procesa tecla liberada
- `Colision_proyectil(proyectil_proyectil*): void` - resuelve colisiones de proyectiles
- `limpiar_listas(): void` - elimina objetos destruidos
- `pausar_bool(): bool` - retorna el estado de pausa

Relaciones: Agrega, mueve y elimina todos los objetos del juego. Se comunica con `GameWidget` para actualizar pantalla y con `Enemigos` para la gestión de aliens.



Figura 5: Diagrama de la clase GameLogic

Enemigos Rol: Clase especializada en la gestión y creación de los enemigos del juego.

Responsabilidades:

- Crear las diferentes filas de aliens (Pulpo, Cangrejo, Calamar)

-
- Gestionar las listas de aliens activos
 - Coordinar el movimiento grupal de los enemigos
 - Actualizar el estado de los aliens

Atributos:

- `lista_aliens: QLists<Alien*>` - lista de todos los aliens activos
- `direccion_derecha: bool` - dirección del movimiento grupal

Métodos:

- `Crear_fila_pulpo(): void` - genera fila de pulpos
- `Crear_fila_cangrejo(): void` - genera fila de cangrejos
- `Crear_fila_calamar(): void` - genera fila de calamares
- `Dibujar(Qpainter&): void` - renderiza todos los aliens
- `mover_enemigos(): void` - actualiza posiciones
- `actualizar(): void` - actualiza estado general

3.2.3. Clases de Entidades del Juego

Alien Clase base abstracta que define el comportamiento común de todos los enemigos.

Rol: Clase base que representa a cualquier enemigo del juego.

Responsabilidad: Establecer la estructura general de los aliens: posición, vida, velocidad y forma de dibujar/disparar.

Atributos:

- Heredados de `Dibujable_Abtracto`: `vida`, `x`, `y`, `ancho`, `alto`
- Heredados de `Movil`: `Vx`, `Vy`
- Heredados de `Colisionable`: `colisionado`
- `expresion_alien: int` - identificador del tipo de alien

Métodos:

- `Dibujar(QPainter&): void*` - renderiza el alien (virtual)
- `movimiento(): void*` - actualiza posición según velocidades
- `Disparar(): proyectil*` - genera proyectil enemigo
- `get_area(): QRectF*` - retorna área de colisión

Relaciones:

- Herencia simple hacia: `Calamar`, `Cangrejo`, `Pulpo`
- Composición: Es administrado por `GameLogic` y `Enemigos`

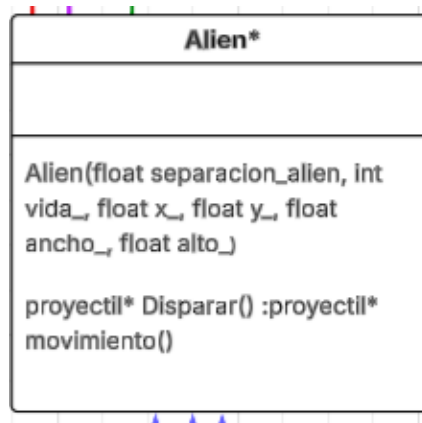


Figura 6: Clase Alien

Clases derivadas: Pulpo, Cangrejo, Calamar Cada tipo de enemigo hereda de Alien y redefine sus características principales:

Pulpo:

- Valor: 10 puntos por eliminación
- Posición: Filas inferiores
- Sprite específico con animación característica

Cangrejo:

- Valor: 20 puntos por eliminación
- Posición: Filas intermedias
- Sprite específico con animación característica

Calamar:

- Valor: 30 puntos por eliminación
- Posición: Filas superiores
- Sprite específico con animación característica

Cada clase sobrescribe el método Dibujar() para renderizar su sprite específico y puede ajustar valores de velocidad o comportamiento según el nivel de dificultad.

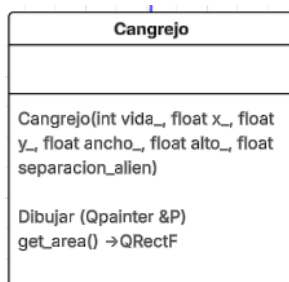


Figura 7: Cangrejo

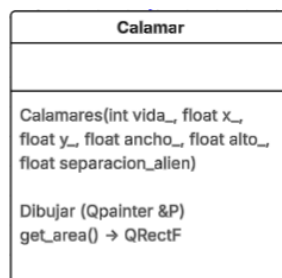


Figura 8: Calamar

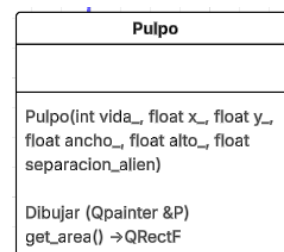


Figura 9: Pulpo

Nave Rol: Representa al jugador, su sprite, movimiento y sistema de vidas.

Responsabilidad: Controlarse horizontalmente mediante input del usuario y disparar proyectiles ascendentes.

Atributos:

- Hereda de Dibujable_Abstracto: vida, x, y, ancho, alto
- Hereda de Movil: Vx, Vy
- Hereda de Colisionable: colisionado
- vidas_iniciales: int = 3 - cantidad de vidas al inicio

Métodos:

- movimiento(): void* - actualiza posición horizontal
- Dibujar(QPainter&): void* - renderiza la nave
- get_area(): QRectF* - retorna área de colisión
- Disparar(): proyectil* - genera proyectil del jugador
- recibir_danio(): void - reduce vidas al recibir impacto
- esta_destruida(): bool - verifica si perdió todas las vidas

Relaciones: Usada y administrada por GameLogic. Genera objetos Proyectil.

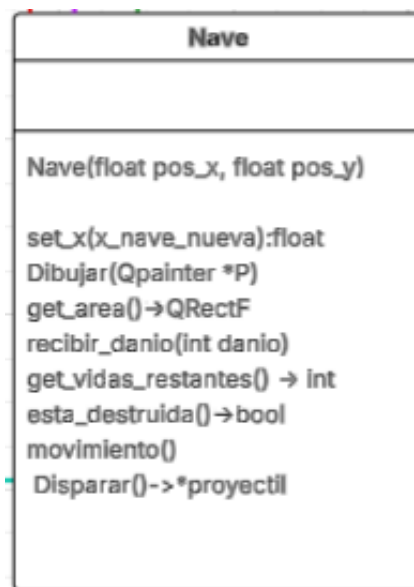


Figura 10: Clase Nave del jugador

Proyectil Rol: Representa los disparos tanto del jugador como de los enemigos.

Responsabilidad: Moverse verticalmente, impactar objetos y dibujarse en pantalla. Auto-eliminarse al salir de los límites de la pantalla.

Atributos:

- Hereda de Dibujable_Abstracto: x, y, ancho, alto
- Hereda de Movil: Vx, Vy
- Hereda de Colisionable: colisionado
- velocidad: float - rapidez del movimiento vertical

Métodos:

- movimiento(): void* - actualiza posición vertical usando Vy
- Dibujar(QPainter&): void* - renderiza el proyectil
- get_area(): QRectF* - retorna área de colisión

Características:

- Los proyectiles del jugador se mueven hacia arriba ($V_y < 0$)
- Los proyectiles enemigos se mueven hacia abajo ($V_y > 0$)
- Dimensiones pequeñas para facilitar la jugabilidad

Relaciones: Instanciado por: Alien::Disparar() y Nave::Disparar(). Administrado por GameLogic.

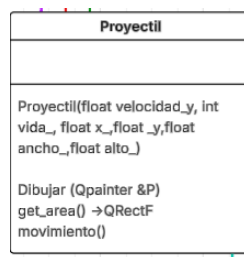


Figura 11: Clase Proyectil

Barrera Rol: Bloque de protección que absorbe proyectiles y se degrada con cada impacto.

Responsabilidad: Representar una estructura degradable entre el jugador y los aliens, proporcionando protección temporal.

Atributos:

- Hereda de Dibujable_Abstracto: vida, x, y, ancho, alto
- Hereda de Colisionable: colisionado
- resistencia: int - cantidad de impactos que puede soportar

Métodos:

- Dibujar(QPainter&): void* - renderiza la barrera con estado visual según daño
- get_area(): QRectF* - retorna área de colisión
- recibir_danio(): void - reduce resistencia y actualiza apariencia

Características:

- Se posiciona estratégicamente entre la nave y los aliens
- Cambia su apariencia visual al recibir daño
- Se destruye completamente al perder toda su resistencia
- No es móvil (no hereda de `Movil`)

Relaciones: Creada, dibujada y removida por `GameLogic`.



Figura 12: Clase Barrera

3.2.4. Nave_enemiga

Rol: Enemigo especial que aparece ocasionalmente en la parte superior.

Responsabilidad: Moverse horizontalmente, desaparecer y otorgar mayor puntaje.

Atributos:

- Dirección
- Velocidad independiente
- Vale 100 puntos

Métodos:

- `movimiento()`
- `fuera_de_pantalla()`
- `Dibujar()`
- `get_area()`

Relaciones: Generada aleatoriamente en `GameLogic`.

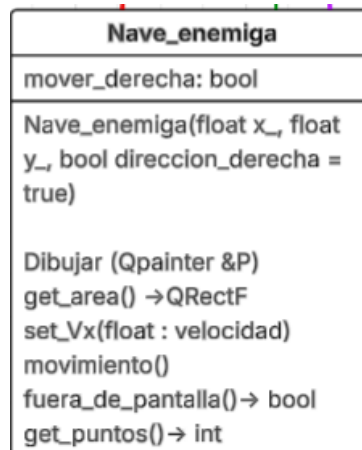


Figura 13: Clase de la nave enemiga

4. Interacciones entre Clases

El ciclo de ejecución del juego sigue el siguiente flujo:

1. Lectura del input del usuario (movimiento y disparo)
2. Actualización del estado de la nave
3. Movimiento de los enemigos
4. Actualización de proyectiles
5. Detección de colisiones:
 - Proyectil–Alien
 - Proyectil–Barrera
 - Alien–Barrera
 - Alien–Nave
6. Actualización del puntaje
7. Renderizado de la escena

5. Justificación del Diseño

El diseño adoptado sigue los principios de la Programación Orientada a Objetos:

- **Modularidad:** Cada entidad del juego está encapsulada en su propia clase
- **Herencia:** Evita duplicación de código para enemigos que comparten comportamiento
- **Agregación y composición:** GameLogic controla el conjunto de objetos sin crear dependencias rígidas

-
- **Escalabilidad:** Permite agregar nuevos tipos de enemigos o modificaciones sin alterar la estructura base
 - **Mantenibilidad:** El sistema puede ser extendido y corregido fácilmente

6. Conclusión

El diagrama UML presentado permite visualizar de manera clara la arquitectura del videojuego Space Invaders. La estructura de clases elegida ofrece un diseño robusto, escalable y coherente con los principios de ingeniería de software. Esto facilita futuras expansiones como nuevos niveles, animaciones adicionales o enemigos alternativos.